Operador — agente que opera sistemas sin API, solo por pantalla

Reto 2 · Frontier CDMX · 31 oct 2026 · Jesús Acevedo · Diagrama de arquitectura adjunto (C4 nivel 2 + zoom al worker) · v2 tras revisión independiente

1 · Problema, supuestos y casos difíciles

Entrada: tarea en español + escritorio (`wss://…/websockify` + contraseña). Solo vemos **pixeles 1280×800 por VNC**: sin DOM, sin API, sin red hacia fuera. **Salida:** la app con los cambios pedidos (más los tolerados en `lista_blanca`), `estado` honesto y `resumen` con formato. Se evalúa en una **app no vista**, escritorio limpio y distinto por tarea, 5 min por tarea, todo-o-nada.

Reglas del contrato que gobiernan el reporte: pregunta → una sola línea `respuesta: <valor>` con el número primero; dos datos → `respuesta: d1; d2` en el orden preguntado; pregunta no respondible → `fallida` **sin** línea `respuesta:`; tarea imposible/peligrosa → `fallida` nombrando los datos; cualquier secreto en el resumen anula la tarea; operar tras reportar contamina la tarea siguiente.

Supuestos (verificables en la app de pruebas): apps web de gestión tipo CRUD en Chromium kiosko (Tab/Enter/Esc funcionan); la fecha "hoy" es visible en la pantalla principal (si no, la tarea con fechas relativas se reporta `fallida` diciendo por qué); texto ≥11 px legible por OCR; el evaluador manda tareas de forma secuencial (si llegan en paralelo, hasta 3 contextos simultáneos).

Casos difíciles que el diseño ataca:

- **Ambigüedad** (dos "Ana Torres") → no adivinar: `fallida` con ambas cédulas.
- **Duplicados por repintado lento** (≤1 s): una captura temprana muestra la pantalla vieja y provoca un segundo "Guardar".
- **Mañas de entrada:** clic <100 ms se ignora; escribir sobre texto concatena; **tildes/ñ** no viajan por el teclado US de Playwright (hay que enviar keysyms RFB).
- **Teclas peligrosas:** Ctrl+W, F5, Alt+F4 cierran o recargan el kiosko → lista negra.
- **Preguntas con muchos datos** (45 facturas → paginación o filtros + total); **fechas** (E4: `<input type=date>` por segmentos, pickers) y fechas relativas desde el "hoy" de la app.
- **Avisos, notas y modales** de confirmación que cambian qué es correcto.
- **Tiempo:** cada paso cuesta 6–13 s (captura + modelo + repintado) → el tope de pasos se deriva del tiempo, no al revés.

Fuera de alcance (a propósito): memorizar SIGECOM (la evaluación usa otra app → sobreajuste); caché de "manuales" entre tareas (riesgo con apps que cambian en el día; se reorienta en cada tarea); velocidad máxima (un fallo honesto cuesta menos que un falso éxito).

2 · Arquitectura (ver diagrama)

- **API** (Node/TS, Fastify, Bearer): `POST /tasks` solo encola y responde <2 s; `GET /tasks/{id}` → `en_curso|terminada|fallida` + resumen (`""` mientras corre). Estado **persistido en SQLite**: un reinicio del contenedor no da 404. `/health` responde 200 solo con Chromium lanzado y key cargada.
- **Worker**: deadline = `recibido_en + 4:30` con abort duro (cancela la llamada al modelo y cierra la página). Fases: **Orientar** (primera captura; buscar "hoy" solo si la tarea trae fechas) → **Planear** (pasos + criterio de éxito; ¿ambigua/peligrosa? → fallar) → **Bucle** observar→decidir→guardias→actuar (≈20–25 pasos) → **Verificar** (releer el registro; en conteos, usar "1–20 de 45"/total del filtro; en exports, el aviso de descarga) → **Reportar** y desconectar VNC.
- **Conector**: Chromium headless (Playwright) con **página propia mínima** que importa solo `core/rfb.js` de noVNC (cliente VNC, no agente): viewport exacto 1280×800, `deviceScaleFactor 1`, sin escalar/recortar; al conectar se **verifica** que el canvas mide 1280×800 en (0,0) → coordenadas 1:1. Captura = framebuffer del canvas. Clic = down-200 ms-up. **Texto = `rfb.sendKey(keysym)` por carácter** (tildes/ñ). `qualityLevel 9` para que el OCR lea dígitos. Lista negra de teclas.
- **Percepción**: captura estable = diferencia <0.2 % de pixeles entre dos cuadros (tolera el cursor parpadeante), máx 1.5 s. **OCR bajo demanda** como herramienta del modelo (`leer_texto` / `buscar_texto`: Tesseract.js, zoom ×2) para cifras exactas e ids; no en cada paso. Solo las 2–3 últimas capturas en el contexto.
- **Cerebro**: Claude con visión y acciones estilo *computer use* (`screenshot·left_click·double_click·mouse_move·type·key·scroll·wait` + `terminar` / `fallar`). **Opción A (por defecto, verificada hoy):** API directa de Anthropic, **Claude Sonnet 5.5** con el toolset nativo `computer_toolset_20260801` (se declara sin medidas); en el spike devolvió `left_click [640,400]` + `screenshot` en un solo turno (agrupa acciones). Costo fijo ≈4.5k tokens de entrada por paso por el prompt interno del toolset → se mide. **Opción B (verificada):** OpenRouter con la key del evento y una **función común `computer`** con el mismo esquema (Sonnet 4.5 respondió `left_click [640,400]`, US$0.003). OpenRouter **rechaza** la tool nativa (400). Misma interfaz interna; solo cambia el adaptador.
- **Guardias en código** (no solo prompt): no repetir "Guardar" sin ver un cambio; tope de pasos, tiempo y $ por tarea; formato de `respuesta:` (una línea, número primero, se elimina si `fallida`); la contraseña nunca entra al prompt + filtro final de contraseña, host del wss y token.

3 · Decisiones y alternativas descartadas

Decisión	Descartamos	Por qué
Visión + OCR bajo demanda	DOM; solo visión; OCR en cada paso	DOM no existe. Solo visión lee mal cifras. OCR en cada paso cuesta 1–3 s × paso.
Playwright + `core/rfb.js` propio	vnc.html de noVNC; libs RFB de Node (`rfb2`); RFB a mano	vnc.html centra el canvas y pinta banners (coordenadas no 1:1). Libs Node sin mantenimiento ni wss. RFB a mano es riesgo alto en 1 día.
Acciones estilo computer use	Formato JSON inventado	Claude está entrenado en ese vocabulario → mejores clics.
API directa + toolset nativo (A)	OpenRouter + función común (B)	Formato en el que el modelo fue entrenado y agrupa acciones por turno; OpenRouter rechaza la tool nativa. B queda como adaptador alterno con la key del evento.
Claude Sonnet 5.5	Opus 5.5 (lento/caro); Haiku 5.5 (peor en clics)	Equilibrio latencia/precisión; se mide en el arnés y se cambia si falla.
Ciclo propio	Mastra/frameworks; servicios tipo Operator	Control de tiempo, pasos y costo. Los servicios que operan solos están prohibidos.
Ante duda, fallar	Adivinar el más probable	Falso éxito y acción destructiva penalizan más que una fallida.
Captura nativa 1280×800	Reducirla	1.02 MP está bajo el umbral en que la API reescala → mismo espacio de coordenadas; el costo lo manda el historial, no la imagen.
SQLite + Railway (1 contenedor)	Estado en memoria; Fly	Sobrevive reinicios; Railway ya lo usamos hoy.

4 · Plan para medir (antes de gastar corridas oficiales)

Smoke tests del conector (primero): teclear "Papelería Los Andes" y leerla en pantalla; canvas 1280×800 en (0,0); clic en una coordenada conocida; latencia por paso.

Arnés local, por caso: `POST /__reset` → `POST /tasks` (mide <2 s) → consultar cada 5 s ≤5 min → `POST /__check` + chequeos propios que `__check` no ve: estado esperado, primer número de `respuesta:`, `claves_razon`, ausencia de secretos, **contador de acciones del conector tras el estado final = 0**.

Casos: E1–E5 + 6 propios (con su JSON e ids reales de SIGECOM): pregunta de 2 datos, fecha relativa, registro inexistente → fallida, modal de confirmación, export con rango de fechas, orden peligrosa → fallida.

Arnés contra el defecto: agente nulo que dice "terminada" (E1 y E5 deben dar ✗); E1 dos veces sin reset (duplicado ✗); resumen con la contraseña (✗).

Métrica	Meta
Éxito E1–E5 · casos propios	≥4/5 · ≥5/6
Falsos éxitos · acciones destructivas · duplicados	0 · 0 · 0
Latencia por paso p50/p95 · pasos por tarea · tiempo p95	medidos · ≤25 · <4 min
OCR en dígitos (ids E5, total E3) · POST /tasks	≥99 % · <2 s
Costo por tarea	medido; tope duro configurado

Compuerta: una corrida oficial solo cuando la versión **desplegada** cumple las metas. 1ª ~16:00, 2ª ~18:00, 3ª de reserva.

5 · Riesgos y mitigación

- Modelo lento → acciones compuestas (Tab entre campos) y tope de pasos derivado del tiempo medido.
- Clics imprecisos → `buscar_texto` OCR para apuntar al control y verificación tras cada acción.
- La app cambia en el día → nada codificado; reorientación en cada tarea.
- Presupuesto (US$45) → tope por tarea y log de costo; capturas podadas del historial.